Only Control S.A. Technological Solutions	**DOCUMENTACIÓN TÉCNICA** **API DE TASAS – TRAFFIC CONTROL** Servicio web REST para el registro y la consulta de tasas desde sistemas externos (ERP)	Fecha de elaboración: **31/10/2026** Versión del documento: **1.0**

1. INFORMACIÓN GENERAL

Producto: Traffic Control – API de Tasas	**Empresa:** ONLY CONTROL S.A.
Tipo de servicio: Servicio web REST (HTTP + JSON)	**Versión de la API:** v1
Publicación: Microsoft IIS sobre Windows Server	**Plataforma:** ASP.NET Core 8 (.NET 8)
Elaborado por: Tnlg. César Cevallos – Only Control S.A.	**Fecha de elaboración:** 31/10/2026

Control de versiones

Versión	Fecha	Descripción	Elaborado por
1.0	31/10/2026	Versión genérica para consumidores del servicio: detalle de servicios, autenticación y ejemplos de consumo.	Tnlg. César Cevallos

Contenido

1. INFORMACIÓN GENERAL 1
   Control de versiones 1
   Contenido 1
2. DESCRIPCIÓN DEL SERVICIO 3
   2.1 ¿Qué es la API de Tasas? 3
   2.2 Características del servicio 3
   2.3 Servicios disponibles 3
3. AUTENTICACIÓN: API KEY 4
   3.1 ¿Qué es la API Key? 4
   3.2 ¿Cómo se envía? 4
   3.3 Cabeceras de cada petición 4
   3.4 Ejemplo de petición autenticada 5
   3.5 Respuestas cuando la autenticación falla 5
4. FORMATO DE LAS RESPUESTAS 6
   4.1 Estructura estándar 6
   4.2 Campo idTransaccion 6
   4.3 Códigos HTTP 7
5. SERVICIO 1: REGISTRAR TASA 7

5.1 ¿Para qué sirve? 7
5.2 ¿Cómo se usa? 7
5.3 Datos de entrada (request) 7
5.4 Ejemplo de consumo 8
5.5 Respuestas de error 9
5.6 Validaciones del servicio 11
6. SERVICIO 2: CONSULTAR ESTADO DE TASA 12
6.1 ¿Para qué sirve? 12
6.2 ¿Cómo se usa? 12
6.3 Estados de la tasa 12
6.4 Ejemplo de consumo 12
6.5 Respuestas de error 13
7. SERVICIO 3: SEÑAL DE VIDA 14
7.1 ¿Para qué sirve? 14
7.2 ¿Cómo se usa? 14
7.3 Ejemplo de consumo 14
8. SERVICIO 4: PRUEBA DE BASE DE DATOS 15
8.1 ¿Para qué sirve? 15
8.2 ¿Cómo se usa? 15
8.3 Ejemplo de consumo 16
9. CÓDIGOS DE RESPUESTA 16
10. RECOMENDACIONES DE CONSUMO 17
11. EJEMPLO DE CONSUMO EN C# 18
12. SOPORTE 18

2. DESCRIPCIÓN DEL SERVICIO

2.1 ¿Qué es la API de Tasas?

La API de Tasas de Traffic Control es un servicio web que permite a un sistema externo, normalmente el ERP de la terminal o de la entidad que emite las tasas, enviar las tasas generadas por las cooperativas a Traffic Control y consultar su estado (si la tasa ya fue utilizada o no).

Traffic Control es el sistema de control de acceso y operación de la terminal. Cuando una tasa se registra a través de esta API, queda disponible en Traffic Control para su uso en el terminal; cuando la tasa se utiliza, su estado cambia y puede consultarse por el mismo servicio.

Con la API el sistema externo puede:

- **Registrar tasas** en Traffic Control, con validación de datos y control de duplicidad.
- **Consultar el estado** de una tasa: no utilizada o utilizada, con su fecha de uso.
- **Verificar que el servicio esté en línea** y que tenga conexión con su base de datos.

Cada transacción queda registrada en un log de auditoría de Traffic Control con la fecha, hora, usuario, IP de origen y resultado.

2.2 Características del servicio

Característica	Detalle
Tipo de servicio	API REST. Se consume con peticiones HTTP estándar (GET y POST).
Formato de datos	JSON con codificación UTF-8, tanto en el envío como en la respuesta.
Publicación	Publicado como sitio web en Microsoft IIS (Internet Information Services) del servidor de Traffic Control.
URL base	La entrega Only Control para cada instalación, con la forma `http://{servidor}:{puerto}`. Ejemplo: `http://192.168.5.2:8085`
Versión	La versión va en la ruta de cada servicio: `/api/v1/...`
Autenticación	API Key enviada en la cabecera HTTP `X-API-KEY` (ver sección 3).
Fechas	Formato ISO `yyyy-MM-ddTHH:mm:ss`, hora local de Ecuador (UTC-05:00).
Respuesta	Todos los servicios responden con la misma estructura JSON (ver sección 4).
Documentación interactiva	Swagger, disponible en la URL base cuando está habilitado en el ambiente de pruebas.

2.3 Servicios disponibles

La API publica cuatro servicios. La URL completa de cada uno se forma con la URL base más la ruta del servicio:

N.º	Servicio	Método	URL	Para qué sirve	API Key
1	Registrar tasa	`POST`	`{URL base}/api/v1/tasas`	Envía una tasa nueva a Traffic Control para que quede registrada y disponible para su uso.	Sí

N.º	Servicio	Método	URL	Para qué sirve	API Key
2	Consultar estado de tasa	GET	{URL base}/api/v1/tasas/{codigoTasa}/estado	Indica si una tasa existe y si ya fue utilizada, con su fecha de uso.	Sí
3	Señal de vida	GET	{URL base}/api/v1/health	Confirma que el servicio está en línea. Para monitoreo.	No
4	Prueba de base de datos	GET	{URL base}/api/v1/health/db	Confirma que el servicio tiene conexión con la base de datos de Traffic Control.	Sí

Ejemplo con la URL base `http://192.168.5.2:8085`:

```
POST  http://192.168.5.2:8085/api/v1/tasas
GET   http://192.168.5.2:8085/api/v1/tasas/20261031000000000002/estado
GET   http://192.168.5.2:8085/api/v1/health
GET   http://192.168.5.2:8085/api/v1/health/db
```

3. AUTENTICACIÓN: API KEY

3.1 ¿Qué es la API Key?

La API Key es una clave secreta que identifica al sistema autorizado a consumir la API. Only Control la genera y la entrega a cada cliente por un canal seguro, separado de este documento.

Sin una API Key válida, los servicios protegidos rechazan la petición. Solo el servicio Señal de vida puede consumirse sin clave.

3.2 ¿Cómo se envía?

La clave se envía en cada petición, en la cabecera HTTP `X-API-KEY`, exactamente como fue entregada:

```
X-API-KEY: <API_KEY>
```

Regla	Detalle
Nombre de la cabecera	X-API-KEY
Valor	La clave entregada por Only Control, tal cual, sin espacios antes ni después.
Frecuencia	En todas las peticiones a servicios protegidos. No hay inicio de sesión ni token que expire.
Dónde NO enviarla	Nunca en la URL ni como parámetro de consulta, ni dentro del JSON.

3.3 Cabeceras de cada petición

Cabecera	Valor	Cuándo se envía
X-API-KEY	<API_KEY>	Siempre, en los servicios protegidos.
Content-Type	application/json	Cuando la petición lleva cuerpo JSON (Registrar tasa).
Accept	application/json	Recomendado en todas las peticiones.

3.4 Ejemplo de petición autenticada

```
GET /api/v1/tasas/20261031000000000002/estado HTTP/1.1
Host: 192.168.5.2:8085
Accept: application/json
X-API-KEY: <API_KEY>
```

3.5 Respuestas cuando la autenticación falla

Situación	HTTP	Código
No se envió la cabecera `X-API-KEY`	401	`API_KEY_REQUERIDA`
La clave no es válida	401	`API_KEY_INVALIDA`
10 claves inválidas seguidas desde la misma IP: la IP se bloquea 15 minutos	429	`IP_BLOQUEADA`
La IP de origen no está autorizada (cuando la instalación restringe IPs)	403	`IP_NO_PERMITIDA`

401 – Sin API Key:

```
HTTP/1.1 401 Unauthorized

{
  "exito": false,
  "codigo": "API_KEY_REQUERIDA",
  "mensaje": "Debe enviar la cabecera X-API-KEY",
  "idTransaccion": "91f23dad7caf40a9a1de499aa5061af1",
  "fechaRespuesta": "2026-10-31T10:32:00",
  "datos": null,
  "errores": []
}
```

401 – API Key inválida:

```
HTTP/1.1 401 Unauthorized

{
  "exito": false,
  "codigo": "API_KEY_INVALIDA",
  "mensaje": "API Key inválida",
  "idTransaccion": "8b494c1e484845389ab92c323d5b57cf",
  "fechaRespuesta": "2026-10-31T10:32:20",
  "datos": null,
  "errores": []
}
```

429 – IP bloqueada por intentos fallidos:

```
HTTP/1.1 429 Too Many Requests

{
  "exito": false,
  "codigo": "IP_BLOQUEADA",
  "mensaje": "Demasiados intentos fallidos. Intente más tarde",
  "idTransaccion": "6f5e4d3c2b1a40f9e8d7c6b5a4f3e2d1",
  "fechaRespuesta": "2026-10-31T10:34:00",
  "datos": null,
  "errores": []
}
```

403 – IP no permitida:

```
HTTP/1.1 403 Forbidden
```

```
{
  "exito": false,
  "codigo": "IP_NO_PERMITIDA",
  "mensaje": "Origen no autorizado",
  "idTransaccion": "1d2c3b4a5f6e47d8c9b0a1f2e3d4c5b6",
  "fechaRespuesta": "2026-10-31T10:33:00",
  "datos": null,
  "errores": []
}
```

> Custodie la API Key como una contraseña. Si sospecha que fue expuesta, solicite a Only Control una nueva: el servicio permite tener dos claves vigentes a la vez, para cambiarla sin interrumpir la integración.

4. FORMATO DE LAS RESPUESTAS

4.1 Estructura estándar

Todos los servicios responden con la misma estructura JSON, tanto si la operación fue exitosa como si hubo un error:

```
{
  "exito": true | false,
  "codigo": "CODIGO_DE_RESULTADO",
  "mensaje": "Descripción legible del resultado",
  "idTransaccion": "Identificador de la transacción",
  "fechaRespuesta": "yyyy-MM-ddTHH:mm:ss",
  "datos": { ... } | null,
  "errores": [ "..." ]
}
```

Campo	Tipo	Descripción
`exito`	boolean	`true` si la operación se completó; `false` si hubo cualquier error.
`codigo`	string	Código del resultado (ver sección 9). Es el campo recomendado para programar la lógica del sistema cliente.
`mensaje`	string	Descripción legible del resultado.
`idTransaccion`	string	Identificador de la transacción (ver 4.2).
`fechaRespuesta`	string (fecha)	Fecha y hora de la respuesta.
`datos`	objeto | null	Información devuelta por el servicio. `null` cuando hay error.
`errores`	array de string	Lista de errores. Vacía (`[]`) en las respuestas exitosas. En validaciones devuelve todos los errores encontrados a la vez.

4.2 Campo idTransaccion

Servicio / resultado	Valor
Registrar tasa – exitoso	ID con el que Traffic Control guardó la tasa.
Consultar estado – tasa encontrada	ID de la tasa en Traffic Control.
Registrar tasa o Consultar estado – con error	`"0"`
Señal de vida, prueba de base de datos, errores de autenticación y de formato	Identificador único (GUID) de la petición.

4.3 Códigos HTTP

HTTP	Significado
`200 OK`	Consulta exitosa.
`201 Created`	Tasa registrada.
`400 Bad Request`	Datos inválidos o JSON mal formado.
`401 Unauthorized`	API Key ausente o inválida.
`403 Forbidden`	IP de origen no autorizada.
`404 Not Found`	La tasa consultada no existe.
`409 Conflict`	La tasa ya está registrada.
`413 Payload Too Large`	El cuerpo de la petición supera 16 KB.
`429 Too Many Requests`	Límite de peticiones superado o IP bloqueada.
`500 Internal Server Error`	Error interno. El mensaje incluye un número de referencia para soporte.
`503 Service Unavailable`	Sin conexión a la base de datos (servicio Prueba de base de datos).

5. SERVICIO 1: REGISTRAR TASA

5.1 ¿Para qué sirve?

Envía a Traffic Control una tasa generada por una cooperativa. El servicio valida los datos, verifica que el código de la tasa no exista, registra la tasa con estado no utilizada y devuelve el ID con el que quedó guardada.

Cuándo usarlo: cada vez que el sistema externo emite una tasa que debe quedar habilitada en Traffic Control. Se envía una tasa por petición.

5.2 ¿Cómo se usa?

Elemento	Detalle
Método	`POST`
URL	`{URL base}/api/v1/tasas` Ejemplo: `http://192.168.5.2:8085/api/v1/tasas`

Cabeceras	`Content-Type: application/Json`
	`Accept: application/json`
	`X-API-KEY: <API_KEY>`

Cuerpo	JSON con los datos de la tasa (ver 5.3).
Respuesta exitosa	`201 Created` con código `TASA_REGISTRADA`.

5.3 Datos de entrada (request)

Campo	Tipo	Formato / longitud	Oblig.	Descripción
`codigoTasa`	string	Solo dígitos, máx. 20	Sí	Código único de la tasa. No puede repetirse. Va como texto para no perder dígitos.
`valorTasa`	number	Máx. 2 decimales	Sí	Valor de la tasa en USD. Mayor a 0. Punto como separador decimal.

Campo	Tipo	Formato / longitud	Oblig.	Descripción
fechaDesde	string	yyyy-MM-ddTHH:mm:ss	Sí	Inicio de vigencia. Menor o igual a fechaHasta.
fechaViaje	string	yyyy-MM-ddTHH:mm:ss	Sí	Fecha y hora del viaje. Entre fechaDesde y fechaHasta.
fechaHasta	string	yyyy-MM-ddTHH:mm:ss	Sí	Fin de vigencia.
idViaje	string	Máx. 50	Sí	Identificador del viaje.
idCooperativa	string	Máx. 50, número entero	Sí	Identificador de la cooperativa. Debe ser un número entero.
idSilo	string	Máx. 50	Sí	Identificador del silo asignado.
usuario	string	Máx. 20	Sí	Usuario del sistema externo que genera la tasa (auditoría).

5.4 Ejemplo de consumo

Request:

```
POST /api/v1/tasas HTTP/1.1
Host: 192.168.5.2:8085
Content-Type: application/json
Accept: application/json
X-API-KEY: <API_KEY>

{
  "codigoTasa": "20261031000000000002",
  "valorTasa": 0.50,
  "fechaDesde": "2026-10-31T00:00:00",
  "fechaViaje": "2026-10-31T14:30:00",
  "fechaHasta": "2026-10-31T23:59:59",
  "idViaje": "VJ-0001",
  "idCooperativa": "15",
  "idSilo": "SILO-01",
  "usuario": "usuario01"
}
```

Response 201 Created – Tasa registrada:

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{
  "exito": true,
  "codigo": "TASA_REGISTRADA",
  "mensaje": "Tasa registrada correctamente",
  "idTransaccion": "15872",
  "fechaRespuesta": "2026-10-31T10:15:03",
  "datos": {
    "codigoTasa": "20261031000000000002",
    "valorTasa": 0.50,
    "fechaDesde": "2026-10-31T00:00:00",
    "fechaHasta": "2026-10-31T23:59:59",
    "idCooperativa": 15,
    "estado": "0",
    "descripcionEstado": "NO UTILIZADA",
    "fechaRegistro": "2026-10-31T10:15:03"
  },
  "errores": []
}
```

Campo de datos	Descripción
codigoTasa	Código de la tasa registrada.

Campo de datos	Descripción
`valorTasa`	Valor registrado, con 2 decimales.
`fechaDesde` / `fechaHasta`	Vigencia registrada.
`idCooperativa`	Cooperativa.
`estado`	`"0"`: registrada, no utilizada.
`descripcionEstado`	`NO UTILIZADA`
`fechaRegistro`	Fecha y hora en que Traffic Control registró la tasa.

cURL:

```
curl -X POST http://192.168.5.2:8085/api/v1/tasas \
  -H "Content-Type: application/json" \
  -H "X-API-KEY: <API_KEY>" \
  -d '{"codigoTasa":"20261031000000000002","valorTasa":0.50,"fechaDesde":"2026-10-31T00:00:00",
      "fechaViaje":"2026-10-31T14:30:00","fechaHasta":"2026-10-31T23:59:59","idViaje":"VJ-0001",
      "idCooperativa":"15","idSilo":"SILO-01","usuario":"usuario01"}'
```

5.5 Respuestas de error

400 – Datos con formato inválido

Se devuelven todos los errores encontrados en una sola respuesta.

Request:

```
{
  "codigoTasa": "ABC123",
  "valorTasa": 0.505,
  "fechaDesde": "2026-10-31 00:00",
  "fechaViaje": "2026-10-31T14:30:00",
  "fechaHasta": "2026-10-31T23:59:59",
  "idViaje": "VJ-0001",
  "idCooperativa": "COOP-15",
  "idSilo": "SILO-01",
  "usuario": "usuario_con_nombre_muy_largo"
}
```

Response:

```
HTTP/1.1 400 Bad Request

{
  "exito": false,
  "codigo": "VALIDACION_ERROR",
  "mensaje": "La tasa contiene errores de validación",
  "idTransaccion": "0",
  "fechaRespuesta": "2026-10-31T10:16:40",
  "datos": null,
  "errores": [
    "CodigoTasa debe ser numérico de máximo 20 dígitos.",
    "ValorTasa admite máximo 2 decimales.",
    "FechaDesde debe tener el formato yyyy-MM-ddTHH:mm:ss.",
    "Usuario excede el máximo de 20 caracteres.",
    "IdCooperativa debe ser un valor numérico entero."
  ]
}
```

400 – Campos obligatorios vacíos

Request:

```
{
```

```
  "codigoTasa": "",
  "valorTasa": null
}
```

Response:

```
HTTP/1.1 400 Bad Request

{
  "exito": false,
  "codigo": "VALIDACION_ERROR",
  "mensaje": "La tasa contiene errores de validación",
  "idTransaccion": "0",
  "fechaRespuesta": "2026-10-31T10:17:30",
  "datos": null,
  "errores": [
    "CodigoTasa es obligatorio.",
    "ValorTasa es obligatorio.",
    "FechaDesde es obligatorio.",
    "FechaViaje es obligatorio.",
    "FechaHasta es obligatorio.",
    "IdViaje es obligatorio.",
    "IdSilo es obligatorio.",
    "Usuario es obligatorio.",
    "IdCooperativa es obligatorio."
  ]
}
```

400 – Fechas inconsistentes

Request:

```
{
  "codigoTasa": "20261031000000000099",
  "valorTasa": 0.50,
  "fechaDesde": "2026-11-01T00:00:00",
  "fechaViaje": "2026-11-05T14:30:00",
  "fechaHasta": "2026-10-31T23:59:59",
  "idViaje": "VJ-0001",
  "idCooperativa": "15",
  "idSilo": "SILO-01",
  "usuario": "usuario01"
}
```

Response:

```
HTTP/1.1 400 Bad Request

{
  "exito": false,
  "codigo": "VALIDACION_ERROR",
  "mensaje": "La tasa contiene errores de validación",
  "idTransaccion": "0",
  "fechaRespuesta": "2026-10-31T10:17:12",
  "datos": null,
  "errores": [
    "FechaDesde no puede ser mayor que FechaHasta."
  ]
}
```

400 – JSON mal formado o con tipos incorrectos

Request (texto en un campo numérico):

```
{
  "codigoTasa": "20261031000000000097",
  "valorTasa": "cincuenta centavos"
}
```

Response:

```
HTTP/1.1 400 Bad Request

{
  "exito": false,
  "codigo": "VALIDACION_ERROR",
  "mensaje": "El JSON enviado no tiene un formato válido",
  "idTransaccion": "3c0f1e9a7b6d4f2e8a1b5c7d9e0f1a2b",
  "fechaRespuesta": "2026-10-31T10:18:05",
  "datos": null,
  "errores": [
    "$.valorTasa: valor inválido o con formato incorrecto",
    "request: valor inválido o con formato incorrecto"
  ]
}
```

409 – Tasa duplicada

El `codigoTasa` ya fue registrado antes (esté utilizada o no). La tasa no se duplica.

```
HTTP/1.1 409 Conflict

{
  "exito": false,
  "codigo": "TASA_DUPLICADA",
  "mensaje": "La tasa 20261031000000000002 ya se encuentra registrada",
  "idTransaccion": "0",
  "fechaRespuesta": "2026-10-31T10:19:21",
  "datos": null,
  "errores": []
}
```

500 – Error interno

```
HTTP/1.1 500 Internal Server Error

{
  "exito": false,
  "codigo": "ERROR_INTERNO",
  "mensaje": "Error interno al procesar la tasa. Referencia: 75e1fcc018584ee79aae75d0e13fd717",
  "idTransaccion": "0",
  "fechaRespuesta": "2026-10-31T10:20:44",
  "datos": null,
  "errores": []
}
```

5.6 Validaciones del servicio

Regla	Mensaje de error
Todos los campos son obligatorios.	`<Campo> es obligatorio.`
`codigoTasa`: solo dígitos, máximo 20.	`CodigoTasa debe ser numérico de máximo 20 dígitos.`
`valorTasa` mayor a 0.	`ValorTasa debe ser mayor a 0.`
`valorTasa` con máximo 2 decimales.	`ValorTasa admite máximo 2 decimales.`
Fechas con formato `yyyy-MM-ddTHH:mm:ss`.	`<Campo> debe tener el formato yyyy-MM-ddTHH:mm:ss.`
`fechaDesde` no mayor que `fechaHasta`.	`FechaDesde no puede ser mayor que FechaHasta.`
`fechaViaje` dentro de la vigencia.	`FechaViaje debe estar entre FechaDesde y FechaHasta.`
Textos dentro de su longitud máxima.	`<Campo> excede el máximo de <n> caracteres.`
`idCooperativa` número entero.	`IdCooperativa debe ser un valor numérico entero.`

Regla	Mensaje de error
`codigoTasa` no registrado previamente.	`409` – `La tasa <código> ya se encuentra registrada`

6. SERVICIO 2: CONSULTAR ESTADO DE TASA

6.1 ¿Para qué sirve?

Devuelve la información de una tasa registrada en Traffic Control y su estado de utilización: si todavía no se usa, o si ya se usó y en qué fecha.

Cuándo usarlo: para conciliar las tasas emitidas con las utilizadas, para confirmar que una tasa quedó registrada (por ejemplo, después de un timeout) o para consultar la fecha de uso.

6.2 ¿Cómo se usa?

Elemento	Detalle
Método	`GET`
URL	`{URL base}/api/v1/tasas/{codigoTasa}/estado` Ejemplo: http://192.168.5.2:8085/api/v1/tasas/20261031000000000002/estado
Parámetro	`codigoTasa` (en la URL): código de la tasa, solo dígitos, máximo 20.
Cabeceras	`Accept: application/json` `X-API-KEY: <API_KEY>`
Cuerpo	No lleva.
Respuesta exitosa	`200 OK` con código `TASA_ENCONTRADA`.

6.3 Estados de la tasa

estado	descripcionEstado	Significado	fechaUso
`"0"`	NO UTILIZADA	La tasa está registrada y disponible para su uso.	`Null`
`"1"`	UTILIZADA	La tasa ya fue utilizada en el terminal.	Fecha y hora de uso

6.4 Ejemplo de consumo

Request:

```
GET /api/v1/tasas/20261031000000000002/estado HTTP/1.1
Host: 192.168.5.2:8085
Accept: application/json
X-API-KEY: <API_KEY>
```

Response 200 OK – Tasa no utilizada:

```
HTTP/1.1 200 OK

{
  "exito": true,
  "codigo": "TASA_ENCONTRADA",
  "mensaje": "Tasa NO UTILIZADA",
  "idTransaccion": "15872",
  "fechaRespuesta": "2026-10-31T10:25:00",
  "datos": {
```

```
    "ipMarca": null,
    "codigoTasa": "20261031000000000002",
    "valorTasa": 0.50,
    "fechaDesde": "2026-10-31T00:00:00",
    "fechaHasta": "2026-10-31T23:59:59",
    "idCooperativa": 15,
    "estado": "0",
    "descripcionEstado": "NO UTILIZADA",
    "fechaRegistro": "2026-10-31T10:15:03",
    "fechaUso": null
  },
  "errores": []
}
```


Response 200 OK – Tasa utilizada:


```
HTTP/1.1 200 OK

{
  "exito": true,
  "codigo": "TASA_ENCONTRADA",
  "mensaje": "Tasa UTILIZADA",
  "idTransaccion": "15872",
  "fechaRespuesta": "2026-10-31T16:02:41",
  "datos": {
    "ipMarca": “192.168.0.19”,
    "codigoTasa": "20261031000000000002",
    "valorTasa": 0.50,
    "fechaDesde": "2026-10-31T00:00:00",
    "fechaHasta": "2026-10-31T23:59:59",
    "idCooperativa": 15,
    "estado": "1",
    "descripcionEstado": "UTILIZADA",
    "fechaRegistro": "2026-10-31T10:15:03",
    "fechaUso": "2026-10-31T15:42:10"
  },
  "errores": []
}
```


Campo de datos	Descripción
`ipMarca`	IP del equipo que realizo la validación de la tasa.
`codigoTasa`	Código de la tasa.
`valorTasa`	Valor de la tasa.
`fechaDesde` / `fechaHasta`	Vigencia de la tasa.
`idCooperativa`	Cooperativa.
`estado` / `descripcionEstado`	Estado de la tasa (ver 6.3).
`fechaRegistro`	Fecha en que se registró en Traffic Control.
`fechaUso`	Fecha y hora de uso; `null` si aún no se utiliza.

cURL:

```
curl http://192.168.5.2:8085/api/v1/tasas/20261031000000000002/estado \
  -H "X-API-KEY: <API_KEY>"
```

6.5 Respuestas de error

400 – Código de tasa inválido

El código contiene caracteres que no son dígitos o supera los 20 dígitos.

```
HTTP/1.1 400 Bad Request
```

```
{
  "exito": false,
  "codigo": "VALIDACION_ERROR",
  "mensaje": "CodigoTasa debe ser numérico de máximo 20 dígitos",
  "idTransaccion": "0",
  "fechaRespuesta": "2026-10-31T10:26:10",
  "datos": null,
  "errores": []
}
```

404 – Tasa no existe

```
HTTP/1.1 404 Not Found

{
  "exito": false,
  "codigo": "TASA_NO_EXISTE",
  "mensaje": "La tasa 99999999999999999999 no existe",
  "idTransaccion": "0",
  "fechaRespuesta": "2026-10-31T10:26:45",
  "datos": null,
  "errores": []
}
```

500 – Error interno

```
HTTP/1.1 500 Internal Server Error

{
  "exito": false,
  "codigo": "ERROR_INTERNO",
  "mensaje": "Error interno al consultar la tasa. Referencia: 0a9b8c7d6e5f40312a1b2c3d4e5f6a7b",
  "idTransaccion": "0",
  "fechaRespuesta": "2026-10-31T10:27:02",
  "datos": null,
  "errores": []
}
```

7. SERVICIO 3: SEÑAL DE VIDA

7.1 ¿Para qué sirve?

Confirma que el servicio está publicado y respondiendo. No requiere API Key y no consulta la base de datos, por lo que es el servicio indicado para herramientas de monitoreo y para la primera prueba de conectividad.

7.2 ¿Cómo se usa?

Elemento	Detalle
Método	`GET`
URL	`{URL base}/api/v1/health` Ejemplo: `http://192.168.5.2:8085/api/v1/health`
Cabeceras	No requiere API Key. Opcional: `Accept: application/json`
Respuesta exitosa	`200 OK` con código `API_OK`.

7.3 Ejemplo de consumo

```
GET /api/v1/health HTTP/1.1
Host: 192.168.5.2:8085
```

```
HTTP/1.1 200 OK

{
  "exito": true,
  "codigo": "API_OK",
  "mensaje": "TrafficControl.Tasas.Api en línea",
  "idTransaccion": "a99d518e2876486e8be2bffa5e368379",
  "fechaRespuesta": "2026-10-31T10:30:05",
  "datos": {
    "servicio": "TrafficControl.Tasas.Api",
    "version": "2026.11.4.0",
    "servidor": "TTM-ONLYCONTROL",
    "inicio": "2026-10-31T08:00:12",
    "tiempoActivo": "0.02:29:53"
  },
  "errores": []
}
```

Campo de datos	Descripción
`servicio`	Nombre del servicio.
`version`	Versión publicada.
`servidor`	Servidor que atendió la petición.
`inicio`	Fecha y hora en que inició el servicio.
`tiempoActivo`	Tiempo en línea (`días.horas:minutos:segundos`).

cURL:

```
curl http://192.168.5.2:8085/api/v1/health
```

8. SERVICIO 4: PRUEBA DE BASE DE DATOS

8.1 ¿Para qué sirve?

Confirma que el servicio tiene conexión con la base de datos de Traffic Control. Sirve para diagnosticar: si la Señal de vida responde pero este servicio falla, el problema está en la conexión a la base de datos y no en la publicación.

8.2 ¿Cómo se usa?

Elemento	Detalle
Método	`GET`
URL	`{URL base}/api/v1/health/db` Ejemplo: `http://192.168.5.2:8085/api/v1/health/db`
Cabeceras	`X-API-KEY: <API_KEY>`
Respuesta exitosa	`200 OK` con código `BD_OK`.
Sin conexión	`503 Service Unavailable` con código `BD_ERROR`.

8.3 Ejemplo de consumo

Response 200 OK – Conexión correcta:

```
GET /api/v1/health/db HTTP/1.1
Host: 192.168.5.2:8085
X-API-KEY: <API_KEY>

HTTP/1.1 200 OK

{
  "exito": true,
  "codigo": "BD_OK",
  "mensaje": "Conexión a la base de datos correcta",
  "idTransaccion": "5b2e7c1d9f0a4b3c8d6e2f1a0b9c8d7e",
  "fechaRespuesta": "2026-10-31T10:31:10",
  "datos": {
    "baseDatos": "<NOMBRE_BASE_DATOS>",
    "tiempoRespuesta": "12 ms"
  },
  "errores": []
}
```

Response 503 – Sin conexión:

```
HTTP/1.1 503 Service Unavailable

{
  "exito": false,
  "codigo": "BD_ERROR",
  "mensaje": "No se pudo conectar a la base de datos",
  "idTransaccion": "e4d3c2b1a0f94e8d7c6b5a4f3e2d1c0b",
  "fechaRespuesta": "2026-10-31T10:31:40",
  "datos": {
    "baseDatos": "<NOMBRE_BASE_DATOS>",
    "tiempoRespuesta": "15008 ms"
  },
  "errores": []
}
```

cURL:

```
curl http://192.168.5.2:8085/api/v1/health/db -H "X-API-KEY: <API_KEY>"
```

9. CÓDIGOS DE RESPUESTA

Código	HTTP	Servicio	Significado	Qué hacer
`TASA_REGISTRADA`	201	Registrar	Tasa registrada.	Guardar el `idTransaccion`.
`TASA_DUPLICADA`	409	Registrar	La tasa ya estaba registrada.	No reenviar. Consultar el estado si hace falta.
`TASA_ENCONTRADA`	200	Consultar	Tasa encontrada.	Revisar `estado` y `fechaUso`.
`TASA_NO_EXISTE`	404	Consultar	La tasa no está registrada.	Verificar el código o registrarla.
`VALIDACION_ERROR`	400	Registrar / Consultar	Datos inválidos o JSON mal formado.	Corregir según `errores`. No reintentar igual.
`ERROR_INTERNO`	500	Todos	Error interno.	Reintentar consultando antes el estado; si persiste, reportar la referencia.
`API_OK`	200	Señal de vida	Servicio en línea.	—
`BD_OK`	200	Prueba BD	Conexión correcta.	—

Código	HTTP	Servicio	Significado	Qué hacer
`BD_ERROR`	503	Prueba BD	Sin conexión a la base de datos.	Reportar a soporte.
`API_KEY_REQUERIDA`	401	Protegidos	Falta la cabecera `X-API-KEY`.	Agregar la cabecera.
`API_KEY_INVALIDA`	401	Protegidos	Clave incorrecta.	Verificar la clave.
`IP_BLOQUEADA`	429	Protegidos	IP bloqueada por intentos fallidos.	Corregir la clave y esperar 15 min.
`IP_NO_PERMITIDA`	403	Protegidos	IP no autorizada.	Coordinar la IP con Only Control.
`LIMITE_PETICIONES`	429	Todos	Más de 300 peticiones por segundo desde la IP.	Esperar 1 s (`Retry-After`) y reintentar.
`SOLICITUD_MUY_GRANDE`	413	Registrar	Cuerpo mayor a 16 KB.	Enviar una tasa por petición.

429 – Límite de peticiones:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 1

{
  "exito": false,
  "codigo": "LIMITE_PETICIONES",
  "mensaje": "Se excedió el límite de peticiones. Reintente en 1 segundo",
  "idTransaccion": "c1b2a3f4e5d64c7b8a9f0e1d2c3b4a5f",
  "fechaRespuesta": "2026-10-31T10:35:00",
  "datos": null,
  "errores": []
}
```

413 – Solicitud muy grande:

```
HTTP/1.1 413 Payload Too Large

{
  "exito": false,
  "codigo": "SOLICITUD_MUY_GRANDE",
  "mensaje": "El tamaño de la solicitud excede el máximo permitido",
  "idTransaccion": "7a6b5c4d3e2f41a0b9c8d7e6f5a4b3c2",
  "fechaRespuesta": "2026-10-31T10:36:00",
  "datos": null,
  "errores": []
}
```

10. RECOMENDACIONES DE CONSUMO

- **Una tasa por petición.** Cada tasa se procesa y responde de forma individual.
- **Sin riesgo de duplicar:** el `codigoTasa` es único. Si se reenvía una tasa ya registrada, el servicio responde `409` y no la duplica.
- **Si no llega respuesta** (timeout o corte de red) al registrar, consulte el estado de la tasa antes de reenviarla: `TASA_ENCONTRADA` indica que ya quedó registrada; `TASA_NO_EXISTE`, que debe enviarse otra vez.
- **Programe la lógica con el campo** `codigo`, no con el texto de `mensaje`.
- **Reintentos automáticos** solo para `429 LIMITE_PETICIONES` y `503`, con un máximo de 3 intentos. Los errores `400`, `401`, `403` y `409` no se reintentan: hay que corregir la causa.
- **Timeout recomendado** del cliente HTTP: 30 segundos.

- **Fechas** siempre en hora local de Ecuador y con el formato exacto `yyyy-MM-ddTHH:mm:ss`.
- **Para soporte,** reporte a Only Control el `codigoTasa`, la hora del envío y, en errores `500`, el número de referencia que trae el mensaje.

11. EJEMPLO DE CONSUMO EN C#

Ejemplo breve para .NET 6 o superior, con las librerías incluidas en .NET. Consume la Señal de vida sin API Key y, después de agregar la cabecera `X-API-KEY`, registra una tasa y consulta su estado:

```
// Ejemplo de consumo en C# (.NET 6 o superior)
using System.Net.Http.Json;

var http = new HttpClient
{
    BaseAddress = new Uri("http://192.168.5.2:8085/"),   // URL base entregada por Only Control
    Timeout = TimeSpan.FromSeconds(30)
};

// 1. Señal de vida (no requiere API Key)
var vida = await http.GetAsync("api/v1/health");
Console.WriteLine($"{(int)vida.StatusCode} {await vida.Content.ReadAsStringAsync()}");

// 2. A partir de aquí los servicios requieren la API Key: se agrega una sola vez
http.DefaultRequestHeaders.Add("X-API-KEY", "<API_KEY>");

// 3. Registrar una tasa
var tasa = new
{
    codigoTasa    = "20261031000000000002",
    valorTasa     = 0.50m,
    fechaDesde    = "2026-10-31T00:00:00",
    fechaViaje    = "2026-10-31T14:30:00",
    fechaHasta    = "2026-10-31T23:59:59",
    idViaje       = "VJ-0001",
    idCooperativa = "15",
    idSilo        = "SILO-01",
    usuario       = "usuario01"
};

var registro = await http.PostAsJsonAsync("api/v1/tasas", tasa);
Console.WriteLine($"{(int)registro.StatusCode} {await registro.Content.ReadAsStringAsync()}");

// 4. Consultar el estado de la tasa
var estado = await http.GetAsync($"api/v1/tasas/{tasa.codigoTasa}/estado");
Console.WriteLine($"{(int)estado.StatusCode} {await estado.Content.ReadAsStringAsync()}");
```

Salida esperada (resumida):

```
200 {"exito":true,"codigo":"API_OK","mensaje":"TrafficControl.Tasas.Api en línea",...}
201 {"exito":true,"codigo":"TASA_REGISTRADA","mensaje":"Tasa registrada
correctamente","idTransaccion":"15872",...}
200 {"exito":true,"codigo":"TASA_ENCONTRADA","mensaje":"Tasa NO UTILIZADA","idTransaccion":"15872",...}
```

> En una aplicación real, reutilice una sola instancia de `HttpClient` y guarde la API Key en la configuración segura del sistema, no en el código.

12. SOPORTE

Empresa: ONLY CONTROL S.A. – Technological Solutions	**Responsable técnico:** Tnlg. César Cevallos